How I Use Claude Code

Feb 10, 2026 · [agents sdlc]

Table of Contents

Phase 1: Research

Phase 2: Planning

The Annotation Cycle

Phase 3: Implementation

Feedback During Implementation

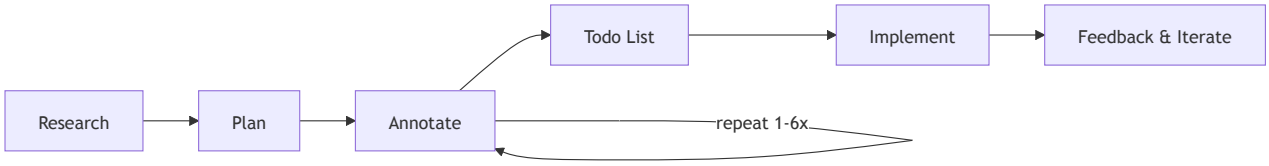
Staying in the Driver's Seat

Single Long Sessions

The Workflow in One Sentence

I've been using Claude Code as my primary development tool for approx 9 months, and the workflow I've settled into is radically different from what most people do with AI coding tools. Most developers type a prompt, sometimes use plan mode, fix the errors, repeat. The more terminally online are stitching together ralph loops, mcps, gas towns (remember those?), etc. The results in both cases are a mess that completely falls apart for anything non-trivial.

The workflow I'm going to describe has one core principle: **never let Claude write code until you've reviewed and approved a written plan**. This separation of planning and execution is the single most important thing I do. It prevents wasted effort, keeps me in control of architecture decisions, and produces significantly better results with minimal token usage than jumping straight to code.



Phase 1: Research

Every meaningful task starts with a deep-read directive. I ask Claude to thoroughly understand the relevant part of the codebase before doing anything else. And I always require the findings to be written into a persistent markdown file, never just a verbal summary in the chat.

> *read this folder in depth, understand how it works deeply, what it does and all its specificities. when that's done, write a detailed report of your learnings and findings in research.md*

> *study the notification system in great details, understand the intricacies of it and write a detailed research.md document with everything there is to know about how notifications work*

> *go through the task scheduling flow, understand it deeply and look for potential bugs. there definitely are bugs in the system as it sometimes runs tasks that should have been cancelled. keep researching the flow until you find all the bugs, don't stop until all the bugs are found. when you're done, write a detailed report of your findings in research.md*

Notice the language: **"deeply"**, **"in great details"**, **"intricacies"**, **"go through everything"**. This isn't fluff. Without these words, Claude will skim. It'll read a file, see what a function does at the signature level, and move on. You need to signal that surface-level reading is not acceptable.

The written artifact (`research.md`) is critical. It's not about making Claude do homework. It's my review surface. I can read it, verify Claude actually understood the system, and correct misunderstandings before any planning happens. If the research is wrong, the plan will be wrong, and the implementation will be wrong. Garbage in, garbage out.

This is the most expensive failure mode with AI-assisted coding, and it's not wrong syntax or bad logic. It's implementations that work in isolation but break the surrounding system. A function that ignores an existing caching layer. A migration that doesn't account for the

ORM's conventions. An API endpoint that duplicates logic that already exists elsewhere. The research phase prevents all of this.

Phase 2: Planning

Once I've reviewed the research, I ask for a detailed implementation plan in a separate markdown file.

> *I want to build a new feature <name and description> that extends the system to perform <business outcome>. write a detailed plan.md document outlining how to implement this. include code snippets*

> *the list endpoint should support cursor-based pagination instead of offset. write a detailed plan.md for how to achieve this. read source files before suggesting changes, base the plan on the actual codebase*

The generated plan always includes a detailed explanation of the approach, code snippets showing the actual changes, file paths that will be modified, and considerations and trade-offs.

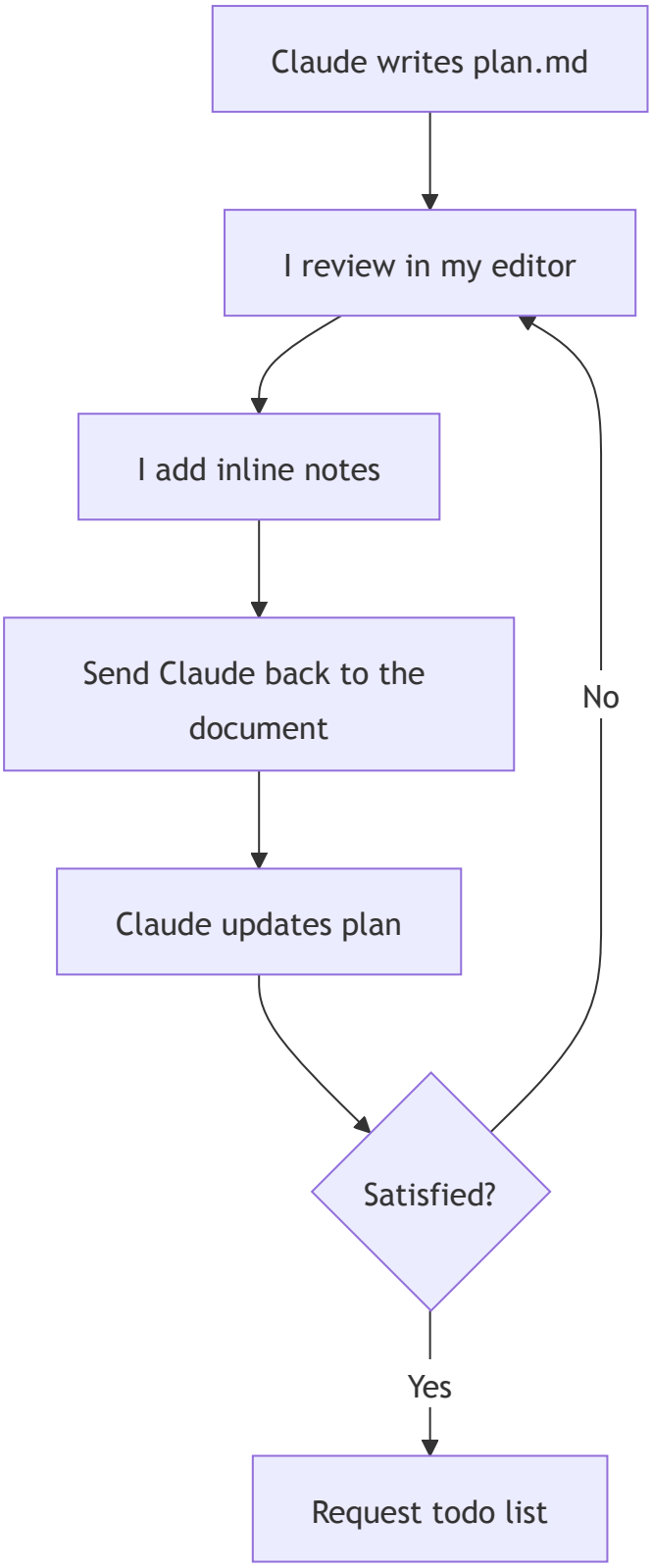
I use my own `.md` plan files rather than Claude Code's built-in plan mode. The built-in plan mode sucks. My markdown file gives me full control. I can edit it in my editor, add inline notes, and it persists as a real artifact in the project.

One trick I use constantly: for well-contained features where I've seen a good implementation in an open source repo, I'll share that code as a reference alongside the plan request. If I want to add sortable IDs, I paste the ID generation code from a project that does it well and say "this is how they do sortable IDs, write a plan.md explaining how we can adopt a similar approach." Claude works dramatically better when it has a concrete reference implementation to work from rather than designing from scratch.

But the plan document itself isn't the interesting part. The interesting part is what happens next.

The Annotation Cycle

This is the most distinctive part of my workflow, and the part where I add the most value.



After Claude writes the plan, I open it in my editor and **add inline notes directly into the document**. These notes correct assumptions, reject approaches, add constraints, or provide domain knowledge that Claude doesn't have.

The notes vary wildly in length. Sometimes a note is two words: "not optional" next to a parameter Claude marked as optional. Other times it's a paragraph explaining a business

constraint or pasting a code snippet showing the data shape I expect.

Some real examples of notes I'd add:

- *"use drizzle:generate for migrations, not raw SQL"* — domain knowledge Claude doesn't have
- *"no — this should be a PATCH, not a PUT"* — correcting a wrong assumption
- *"remove this section entirely, we don't need caching here"* — rejecting a proposed approach
- *"the queue consumer already handles retries, so this retry logic is redundant. remove it and just let it fail"* — explaining why something should change
- *"this is wrong, the visibility field needs to be on the list itself, not on individual items. when a list is public, all items are public. restructure the schema section accordingly"* — redirecting an entire section of the plan

Then I send Claude back to the document:

> *I added a few notes to the document, address all the notes and update the document accordingly. don't implement yet*

This cycle repeats 1 to 6 times. The explicit **"don't implement yet"** guard is essential. Without it, Claude will jump to code the moment it thinks the plan is good enough. It's not good enough until I say it is.

Why This Works So Well

The markdown file acts as **shared mutable state** between me and Claude. I can think at my own pace, annotate precisely where something is wrong, and re-engage without losing context. I'm not trying to explain everything in a chat message. I'm pointing at the exact spot in the document where the issue is and writing my correction right there.

This is fundamentally different from trying to steer implementation through chat messages. The plan is a structured, complete specification I can review holistically. A chat conversation is something I'd have to scroll through to reconstruct decisions. The plan wins every time.

Three rounds of "I added notes, update the plan" can transform a generic implementation plan into one that fits perfectly into the existing system. Claude is excellent at understanding code, proposing solutions, and writing implementations. But it doesn't know my product priorities, my users' pain points, or the engineering trade-offs I'm willing to make. The annotation cycle is how I inject that judgement.

The Todo List

Before implementation starts, I always request a granular task breakdown:

> *add a detailed todo list to the plan, with all the phases and individual tasks necessary to complete the plan - don't implement yet*

This creates a checklist that serves as a progress tracker during implementation. Claude marks items as completed as it goes, so I can glance at the plan at any point and see exactly where things stand. Especially valuable in sessions that run for hours.

Phase 3: Implementation

When the plan is ready, I issue the implementation command. I've refined this into a standard prompt I reuse across sessions:

> *implement it all. when you're done with a task or phase, mark it as completed in the plan document. do not stop until all tasks and phases are completed. do not add unnecessary comments or jsdocs, do not use any or unknown types. continuously run typecheck to make sure you're not introducing new issues.*

This single prompt encodes everything that matters:

- *"implement it all"*: do everything in the plan, don't cherry-pick
- *"mark it as completed in the plan document"*: the plan is the source of truth for progress
- *"do not stop until all tasks and phases are completed"*: don't pause for confirmation mid-flow
- *"do not add unnecessary comments or jsdocs"*: keep the code clean
- *"do not use any or unknown types"*: maintain strict typing
- *"continuously run typecheck"*: catch problems early, not at the end

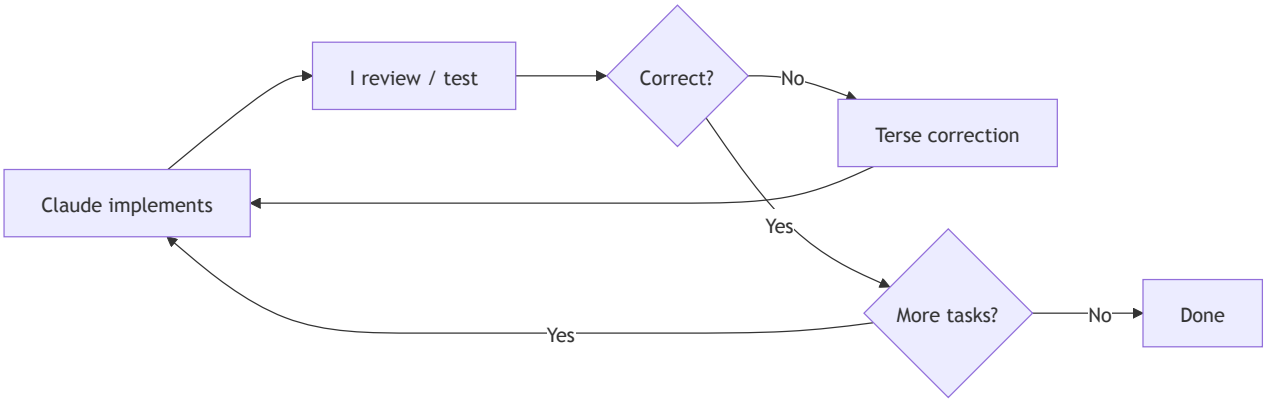
I use this exact phrasing (with minor variations) in virtually every implementation session. By the time I say "implement it all," every decision has been made and validated. The implementation becomes mechanical, not creative. This is deliberate. **I want implementation to be boring**. The creative work happened in the annotation cycles. Once the plan is right, execution should be straightforward.

Without the planning phase, what typically happens is Claude makes a reasonable-but-wrong assumption early on, builds on top of it for 15 minutes, and then I have to unwind a

chain of changes. The “don’t implement yet” guard eliminates this entirely.

Feedback During Implementation

Once Claude is executing the plan, my role shifts from architect to supervisor. My prompts become dramatically shorter.



Where a planning note might be a paragraph, an implementation correction is often a single sentence:

- *“You didn’t implement the `deduplicateByTitle` function.”*
- *“You built the settings page in the main app when it should be in the admin app, move it.”*

Claude has the full context of the plan and the ongoing session, so terse corrections are enough.

Frontend work is the most iterative part. I test in the browser and fire off rapid corrections:

- *“wider”*
- *“still cropped”*
- *“there’s a 2px gap”*

For visual issues, I sometimes attach screenshots. A screenshot of a misaligned table communicates the problem faster than describing it.

I also reference existing code constantly:

- *“this table should look exactly like the users table, same header, same pagination, same row density.”*

This is far more precise than describing a design from scratch. Most features in a mature codebase are variations on existing patterns. A new settings page should look like the existing settings pages. Pointing to the reference communicates all the implicit requirements without spelling them out. Claude would typically read the reference file(s) before making the correction.

When something goes in a wrong direction, I don't try to patch it. I revert and re-scope by discarding the git changes:

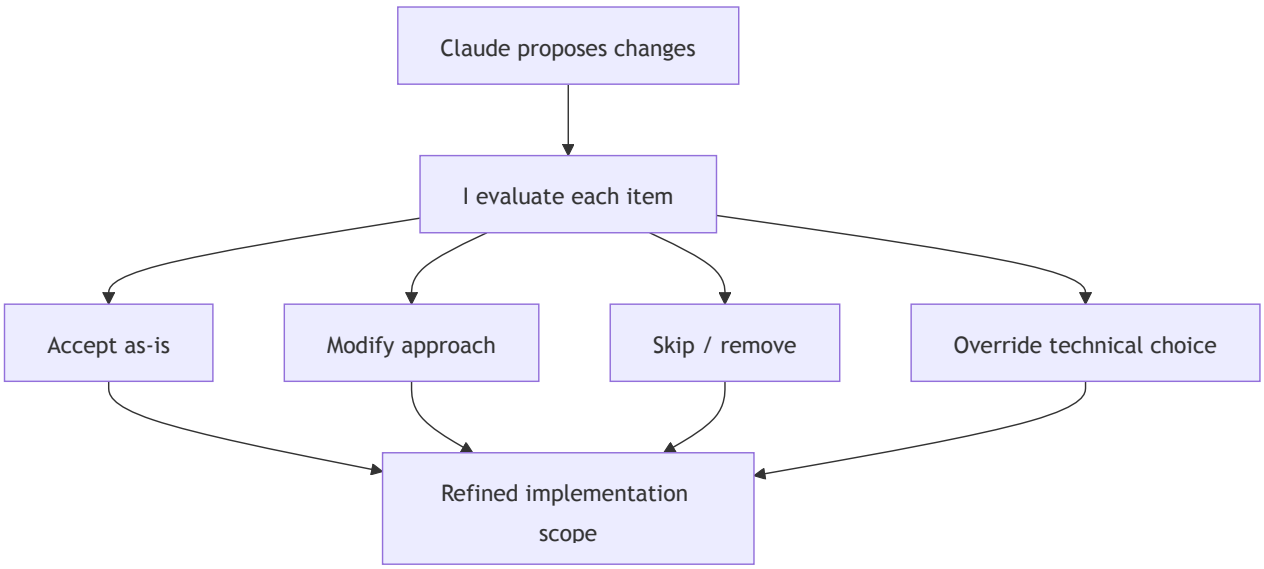
- *"I reverted everything. Now all I want is to make the list view more minimal — nothing else."*

Narrowing scope after a revert almost always produces better results than trying to incrementally fix a bad approach.

Staying in the Driver's Seat

Even though I delegate execution to Claude, **I never give it total autonomy over what gets built**. I do the vast majority of the active steering in the `plan.md` documents.

This matters because Claude will sometimes propose solutions that are technically correct but wrong for the project. Maybe the approach is over-engineered, or it changes a public API signature that other parts of the system depend on, or it picks a more complex option when a simpler one would do. I have context about the broader system, the product direction, and the engineering culture that Claude doesn't.



Cherry-picking from proposals: When Claude identifies multiple issues, I go through them one by one: *"for the first one, just use Promise.all, don't make it overly complicated; for the third one, extract it into a separate function for readability; ignore the fourth and fifth ones, they're not worth the complexity."* I'm making item-level decisions based on my knowledge of what matters right now.

Trimming scope: When the plan includes nice-to-haves, I actively cut them. *"remove the download feature from the plan, I don't want to implement this now."* This prevents scope creep.

Protecting existing interfaces: I set hard constraints when I know something shouldn't change: *"the signatures of these three functions should not change, the caller should adapt, not the library."*

Overriding technical choices: Sometimes I have a specific preference Claude wouldn't know about: *"use this model instead of that one"* or *"use this library's built-in method instead of writing a custom one."* Fast, direct overrides.

Claude handles the mechanical execution, while I make the judgement calls. The plan captures the big decisions upfront, and selective guidance handles the smaller ones that emerge during implementation.

Single Long Sessions

I run research, planning, and implementation in a **single long session** rather than splitting them across separate sessions. A single session might start with deep-reading a folder, go through three rounds of plan annotation, then run the full implementation, all in one continuous conversation.

I am not seeing the performance degradation everyone talks about after 50% context window. Actually, by the time I say "implement it all," Claude has spent the entire session building understanding: reading files during research, refining its mental model during annotation cycles, absorbing my domain knowledge corrections.

When the context window fills up, Claude's auto-compaction maintains enough context to keep going. And the plan document, the persistent artifact, survives compaction in full fidelity. I can point Claude to it at any point in time.

The Workflow in One Sentence

Read deeply, write a plan, annotate the plan until it's right, then let Claude execute the whole thing without stopping, checking types along the way.

That's it. No magic prompts, no elaborate system instructions, no clever hacks. Just a disciplined pipeline that separates thinking from typing. The research prevents Claude from making ignorant changes. The plan prevents it from making wrong changes. The annotation cycle injects my judgement. And the implementation command lets it run without interruption once every decision has been made.

Try my workflow, you'll wonder how you ever shipped anything with coding agents without an annotated plan document sitting between you and the code.

```
I'm building nominal.dev , check it out
share blog: twitter mail
grep [ agents  sdlc ]
goto ship-types
cd ..
```

Join my newsletter to be the first to know about new blog posts :)

you@website.com

Subscribe

---

48 Comments - *powered by utteranc.es*

abeixiaolu commented 3 weeks ago

Learned a lot, thank you for your selfless sharing.❤️

❤️ 1

Rjshakya commented 3 weeks ago

Thanku, boris. As always very helpful ❤️ .

❤️ 1

GiffenGood commented 3 weeks ago

I presume you feed the research.md to the planning phase?

❤️ 1

dragosroua commented 3 weeks ago

That's basically Assess - Decide - Do, with slightly different names, so I can thoroughly relate. I already integrated ADD into my Claude apps, first as a mega prompt, then as a flexible collection of skills and even a Cowork plugin. Feel free to to have a look at the repos, if you think this is helpful.

❤️ 1

GiffenGood commented 3 weeks ago

I really enjoyed this article. It is largely how I work with claude.

Is there any reason you don't write the results of your research to path specific claude.md files, so that:

1. you can re-use in future sessions
2. you are not wasting tokens researching things over and over again

❤️ 1

wgraham17 commented 3 weeks ago

I've come to similar conclusions and ended up codifying it outside of my IDE/Claude Code (as I've had some spare time lately). Proper steering and gated checkpoints are crucial for anything but the most trivial of tasks.

❤️ 1

JoshuaWellbrock commented 3 weeks ago

Nice article, especially the point that using a plan dramatically improves an agent's output quality. I've seen the same effect in practice.

Many of those "manual" steps are also a perfect fit for skills, where you can enforce things like waiting for explicit human approval before continuing.

That's exactly why we introduced Agile V, an open standard for AI-augmented engineering, along with ready-to-use skills that make planning, approvals, and structured execution easy to adopt. https://github.com/Agile-V/agile-v-skills

https://github.com/Agile-V/agile_v_skills

❤️ 1

boristane commented 3 weeks ago Owner

@GiffenGood

> I presume you feed the research.md to the planning phase?

I don't always, but if in the plan the agent goes in a direction I don't like, I can directly reference the `research.md` file in the notes for the agent and steer it in the right direction :)

boristane commented 3 weeks ago Owner

@GiffenGood

> I really enjoyed this article. It is largely how I work with claude.
>
> Is there any reason you don't write the results of your research to path specific claude.md files, so that:
>
> 1. you can re-use in future sessions
> 2. you are not wasting tokens researching things over and over again

Codebases change rapidly, I don't think persisting instructions permanently is the way forward. I have `CLAUDE.md` files, but they capture only things that are very unlikely to change (frameworks, auth, etc.). I let the agent rediscover everything every time - it's not that costly nor slow.

alamehan commented 3 weeks ago

Just use GSD (https://github.com/gsd-build/get-shit-done). It automates this exact workflow.

👍 2 ❤️ 1

robinunlearning commented 3 weeks ago

Hey Boris. Great article and insight. Im wondering, what do you do with all the research, plan and todo files? Do you integrate them with a backlog management tool for example?

Cheers
Robin

❤️ 1

mcandea04 commented 3 weeks ago

Congrats, you are doing your own flavor of spec-driven development.

👍 1

boristane commented 3 weeks ago Owner

> Hey Boris. Great article and insight. Im wondering, what do you do with all the research, plan and todo files? Do you integrate them with a backlog management tool for example?
>
> Cheers Robin

@robinmatthews at the moment I delete them once I finish the session with the model. However I intent to start saving them alongside the PR such that they can be referenced in the future, or at code review (although I am very light on code review with this workflow).

rowanu commented 3 weeks ago

What's so bad about Claude's plan mode @boristane?

JoanComasFdz commented 3 weeks ago

The superpowers skills are now part of the official Anthropic repository and they do an excellent job, brainstorm for research, write plan for the tasks, execute plan.

I've had a lot of success since I started using them.

On top I use Serena and Context7, it's a great combo.

jbrinkman commented 3 weeks ago

I have long been a proponent of spec driven development. This is just one more variation of BMAD, OpenSpec, Github Spec-kit, etc. The important part of all spec driven development is that we don't forget 80 years of learning on how to write good software. We create user stories to capture the high-level details of what needs to be built. We ask questions about those user stories and update them to ensure we have all the critical details captured. Then we build a design document that takes those business requirements and starts refining it into technical details. Finally, the enginneering team takes a user story and breaks it down into small bite-sized tasks needed to implement the design: create the contracts, write the API, create the UI, wire front-end/backend together, create unit and integration tests, etc. None of this should change just because we are using AI.

❤️ 1

birchwoodmedia commented 3 weeks ago

While this is written for experienced devs, it is good advice for us vibe coders as well. It all rings true for any model/environment. The main deviation that I have, is that I can't spec it completely in advance, if it's a new feature or product, I have to see it and play with it before I can nail down the spec to where I'm happy with the product as a first generation. But even then, the process described here is good for best results, minimal thrashing.
As a side note, while I haven't used Claude Code yet (I've used Cursor a lot, and have downloaded Antigravity, with the intention of trying it for the next "bigger" project), I love Claude online. I copy/paste the artifacts as entire files, if the project has about 6 affected files or less. But, as models go, I'm blown away at how much Claude seems to read my mind, regardless of situation (coding, writing manifestos, solving the world's problems, etc). I can only afford the Pro plan and when I hit the weekly limits and try continuing with chatGPT or DeepSeek (haven't tried Gemini lately, but don't think it's online version is as good as the coding api's in Cursor, eg), I love Claude even more. The difference in quality and communication is dramatic.

btw, I started defining abbreviations in my general .md file for oft used phrases, like "mnc" = "make no changes yet". This file is where I describe my server, my stack, that is the first doc uploaded. Even then, after time passes, it forgets my stack specifics (legacy code restricts to older syntax), and I have to rub its nose in the general.md.

The other main feature of Claude that is way better, is the screenshot capture/share. It would be nice if it could sense tabs, not just windows. But not having to manually make a screenshot, save

👍 1

birchwoodmedia commented 3 weeks ago

I should add, that as a vibe coder, I don't mind copy/pasting every set of changes. It's only 10 secs, max, and I get to have the emergency brake on for every iteration. If the last code Claude generated isn't what I want, I just don't implement it. If Claude gets too ahead of itself, makes decisions that I wouldn't, I'll usually see that when I skim the description of the changes. Or, if I don't see it until I paste/apply/view, I can just ctrlZ the editor, and tell Claude I undid the last changes. My current work is smaller chunks, so I haven't needed to use the .git repository commands for a long time. A lot of that is due to Claude "just getting it" and doing it correctly the first time.
Several months ago, the LLMs were bad for design creep, changing working code in established sections, for no reason, other than their incessant need to change everything. Abandoned functions, etc.
At this moment in time, I feel that Claude is my soulmate.
:)

marcinrabiej commented 3 weeks ago

Thanks Boris - great write-up. I'm trying to map your doc-driven flow onto Claude Code's built-ins and I'm confused about one thing:
In my setup Claude sometimes "wants to" switch into Plan Mode while I'm still shaping the plan/todo. Do you intentionally force Plan Mode for research/planning (read-only), or do you allow normal mode but block execution purely via the "don't implement yet" instruction?

normal mode but block execution purely via the "don't implement yet" instruction?
Also: when you ask for a granular todo list, is it strictly a plan.md checklist, or do you ever want Claude to maintain the built-in Tasks list (Ctrl+T) and execute from that as the source of truth?

TerrellV commented 2 weeks ago

Great article. Are you committing / staging the first version of your plan.md and then also committing/staging your annotations to it? Curious what you are relying on to ensure claude easily picks up on your annotations? I find that I have to at least stage the doc via git before making changes so that the agent can diff the two versions to see what I changed. (maybe this was obvious to some but took me a few failed attempts)

fujohnwang commented 2 weeks ago

I wonder whether you will or ask AI to remove or cross the annotated contents you did in plan.md?

I think after each iteration, the annotated content will be gone, right?

fujohnwang commented 2 weeks ago

Oh, great post, BTW. 🤣

barghi commented 2 weeks ago

It was great
Thank you for sharing this valuable experience

nstpdp commented 2 weeks ago

"I use my own .md plan files rather than Claude Code's built-in plan mode. The built-in plan mode sucks. My markdown file gives me full control. I can edit it in my editor"

All plans are ~/.claude/plans/ in .md so yes you can edit them and separate they are by default

wwj718 commented 2 weeks ago

Unparalleled brilliance!

CookieTheP commented 2 weeks ago

Very nice and useful read
Thanks for sharing!

buildoak commented 2 weeks ago

Been doing roughly the same logic, but with a twist!
I have made so that Claude Code can use codex SDK as subagent in both main session and its Task subagents. Found out several things.

(1) codex 5.3 xhigh is very good auditor for both plans and code written
(2) swarms of codex 5.3 high subagents - are good executors
(3) codex 5.3 spark instead of Haiku - fast and good quality
(4) opus 4.6 always as a final auditor of everything

This leads to much more nasty assumptions being caught, much longer independent operations and much higher quality of code at the result.

Why this works?
(1) Models by different providers seem to have orthogonal mode collapses
(2) opus 4.6 is a very good delegator by nature - so it can run any other subagents

❤️ 1

adamrdrew commented 2 weeks ago

I built a Claude Code plugin that essentially takes this loop and builds it into a set of composable subagents with specific tasks. It also includes a lot more like project governance, style conventions, development phase (think: sprint) records in-repo, and a lot more. Anyone interested in working this way may want to check it out https://github.com/adamrdrew/ushabti

I've been using and iterating-on this for months and it works quite well. I've also built a lot of tooling around it including a claude SDK dev job running using these agents that can kick off dev jobs through a dispatch system, a Mac OS app that allows you to manage and kick off dev jobs for multiple projects in parallel while working at a high level of abstraction. and some tooling that isn't quite done yet but should be public this coming week that has a long running agent with knowledge of this whole stack that can work with the projects, tickets, and more and operate the stack autonomously. I've been doing a lot of dev work just by chatting with my agent on telegram.

🎉 1

sdavid501 commented 2 weeks ago

This feels like a manual operation if the LLM has to repeat this each time. I think a better approach would be creating readme.md files for your repo, and this might be hard to do especially when it comes to updating it each time there's sn updates. This is the approach used for creating MCPs for package repos.

RevinFedor commented 2 weeks ago

Great read, Boris. Your 'Annotation Cycle' is a masterclass in tactical steering-using Markdown as a shared mutable state is far superior to fighting with chat history.

However, I noticed your workflow seems optimized for a single (even if long) session. How do you handle 'project amnesia' across dozens of disconnected sessions over months?

I’ve settled into a 'Post-Session Distillation' phase (part of my Gold Standard v6.0 framework). At the end of every task, I have the agent analyze the session and extract ONLY the 'Invisible Intent'-the motivations, rejected approaches, and platform quirks (like specific Chromium/Electron bugs)-into a permanent docs/knowledge/ and docs/features/ structure.

This effectively turns the documentation into the project's DNA. It makes the 'Research Phase' for the next session almost instant because the agent doesn't have to 'deep-read' the whole codebase every time-it just reads the distilled high-signal context.

Have you explored persistent architectural memory like this, or do you find that re-doing the Research phase each time is a feature, not a bug?

gbro3n commented 2 weeks ago

Thanks Boris. I've been leaning to this flow but your post solidified a lot in my mind. Can I ask - this doesn't align well with the 'running lots of agents' at once trend that I'm seeing with other developers. That's not a flow that works for me anyway - I don't think multitasking lends itself to good work, but have you considered how you might manage your plan documents in such a flow. Also, how large do you let the plan documents get? When do you clear it? I've set up an `AGENTS.md` to instruct my Agents to follow a flow similar to what you have described. I'll post it here in case it's useful to others, comments welcome:

```
# Agent Instruction

Read README.md, this is the main document aimed at both humans and LLMS. Confirm you h

Adhere strictly to these instructions:

- `plan`: If I prompt with 'plan', assume you are to collaboratively plan project task
- `todo`: If I prompt with 'todo', assume you are to update the todo list based.
- `implement`: If I prompt with 'implement', then (and only then) proceed with impleme

I may follow any of these prompts with specific instruction as a means of steering you

## Our Collaborative Workflow

### Planning
```

skillsXchangManager commented 2 weeks ago

Hello, #smarterpeoplethanl ... This is a really informative article. I'm a non-developer, developer. I'm writing an app to help people find jobs in this next wave of a job market. I'm going to try and

use this methodology (and a few others mentioned). I think the problem with this is that I don't know what good looks like. Curious, if I follow this and apply rigor to the design, etc... Can I really assume the output is good (yes I used the Ass word, so I'm sure the answer is no) but how can a dummy like me actually know if something is good or not at the end (quality of code, code resilience/extensibility, etc).

intojhanurag commented 2 weeks ago

thanks borish , It helped a lot.

silverman63 commented a week ago

How do you distinguish your inline comments from what it had written already? Do you have a specific symbol you use for this?

birchwoodmedia commented a week ago

Just have a different AI or two review it. The trick is to know wht is just change for changes sake, and what really needs to be changed for improvement.
...

qishaoyumu commented a week ago

Have you considered designing and writing this workflow as a workflow skill? This would allow more people to use it in a simpler way, and gain a better understanding and useful suggestions in practice.

👍 1

sethatron commented a week ago

I would add one important final step of the plan after you are satisfied with it, asking it something like the following, my comments in parentheses:

```
This looks good! (surprisingly this is helpful, you want to model to know that you do not reject the plan on it's core content) Initiate a deep, comprehensive, and critical (I find this word specifically helps significantly) review of the proposed plan. Identify any gaps, or assumptions built on missing data/context. If applicable (this also helps a lot to prevent it from forcing the matter), update the propsal to include any new observations or material identified during this operation. If you have any final questions on the plan, ask me before finalizing this updated plan.
```

Depending on the complexity of the project, I will sometimes do this 3 or 4 times. It works extremely well, and has often caught issues or suggested improvements that I hadn't even considered

👍 1

Reebz commented a week ago

Consider trying the compound-engineering plugin. I am not affiliated, just a fan. You may find it is aligned to your preferred approach.

Vinix24 commented a week ago

Great writeup Boris. Your annotation cycle is essentially manual quality gates and it works because you're disciplined about it.

I took the same core principle (never let the AI run unchecked) but automated the gates. Single agent per task, codified acceptance criteria per step, execution logs for every decision. No sub-agents — they're a black box when something breaks. The persistent plan.md you describe is the foundation, but I found the real leverage comes when the gates between phases run automatically: if a step fails validation, it doesn't proceed. No "don't implement yet" needed because the system enforces it.

After 6 months in production with this approach (built a full crawler — 26 extractors, 405 tests), the biggest lesson: the governance layer matters more than the planning itself. A mediocre plan with strict gates beats a perfect plan with manual review every time.

Curious how you handle context drift in longer sessions. That's where automated validation saved me the most by catching assumptions that silently shifted after compaction.

Update 28-2-2026: I have opened up my repo at https://github.com/Vinix24/vnx-orchestration for orchestration with governance.

crazygo commented a week ago

Thanks for sharing this workflow!

One friction point: when I comment on plan.md, I read faster than Claude responds. So I'll have multiple comments but get blocked waiting for it to finish before I can send the next one.

How do you handle this? Do you write all your notes into the doc first, then send it to Claude in one go? Or is there a better way?

crazyss commented a week ago

Qoder

Quest Mode
maybe is your friend

f0ghua commented a week ago

Since you implement tasks continuously without pause until all tasks and phases are completed, how do you handle 'Feedback During Implementation'?

minorcell commented a week ago

This looks a lot like CodeX's plan pattern, but CodeX's plan pattern also has its drawbacks, such as being somewhat cumbersome to interact with. However, your method seems more like a general solution.

lianjiexin commented 5 days ago

good tips on having better control. But the more we control, more we fall into deterministic thinking. I have put more focus on testing and monitoring. As long as we have good coverage of tests and monitoring (unit test, integration test, manual tests), I can care less about implementation (not saying I don't want to understand the architecture and implementation, but there is a balance to keep)

minorcell commented 5 days ago

> Have you considered designing and writing this workflow as a workflow skill? This would allow more people to use it in a simpler way, and gain a better understanding and useful suggestions in practice.

Perhaps we don't need to write it as skills; putting it in AGENTS.md/CLAUDE.md should be enough.

25Ericcheong commented 5 days ago

A single long session is surprising to me.

Normally I would separate research, planning and implementation so that the context window doesn't get too large but you're saying its better that way.

But the idea behind continuous reviewing of the implementation plan works! Great article btw

artekr commented 5 days ago

I recently discovered this plugin https://github.com/Langerrr/zforge, it's a mind-blowing workflow. Having /plan → /feature-orchestrate → /review as a coherent pipeline, with persistent docs artifacts that survive context resets, is exactly what's needed for serious feature work.

JoshuaWellbrock commented 5 days ago

We've looked at this topic extensively, especially for companies in regulated environments, and proposed a framework for working with AI agents. You can find the full article on arXiv: https://arxiv.org/abs/2602.20684
We're already working on a few other papers as a follow up to this one (e.g. Mutation Testing and Fuzzing).

ronaldsuwandi commented 3 days ago

Thanks for the article. This is really good. I really like the idea of shared plan where you can directly modify the plan instead of keep asking Claude to modify it through its plan mode.

One quick question once the plan is completely done do you simply clear it and keep the file empty or do you store the old plan as an artifact and renamed it?

Write Preview

Sign in to comment

Styling with Markdown is supported

Sign in with GitHub

bsky github linkedin x (twitter)